

Understanding Microservices Architecture

Concepts to Practice

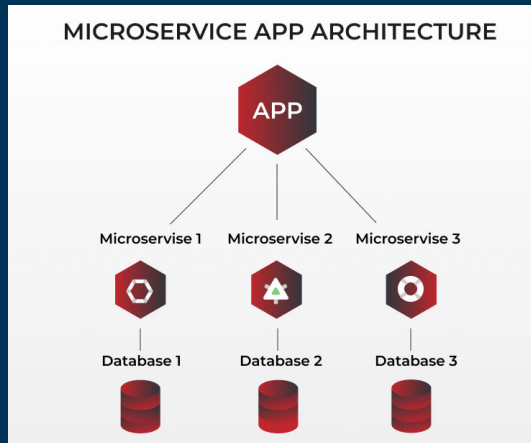
Outline

- 1 Key Statistics on Microservices Adoption
- 2 Microservices Architecture Use Cases
- 3 Choosing Microservices Architecture
- 4 Switching to Microservices Architecture
- 5 Microservices and Security
- 6 Real-world Microservices Use Cases

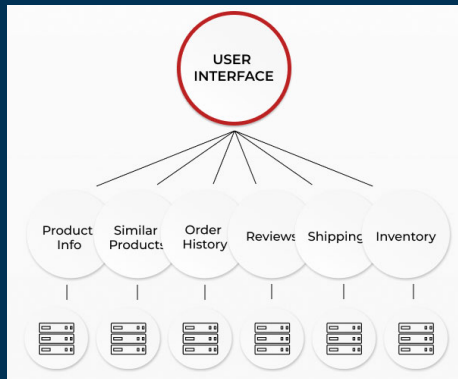
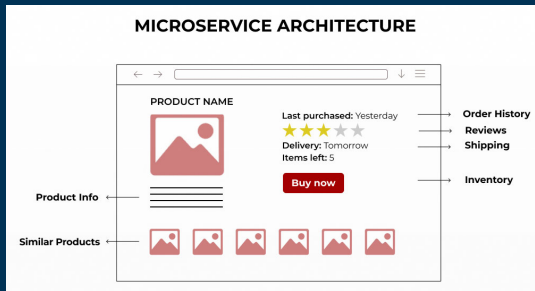
About Microservices

- The microservice architecture foresees the opportunity to build applications that consist of loosely-coupled services.
- Every microservice has a separate database (generally) and can be built using different technologies.
- The key characteristics of microservices-driven apps are:
 - decentralized architecture
 - great scalability
 - failures resistance
- Businesses actively adopt the microservice architecture due to its many benefits. The foremost advantages of using microservices are:
 - self-sufficiency of an application
 - utilization of different technologies
 - apps are easy to scale and update
 - improved continuous delivery
 - simplified developers' onboarding

Microservices Architectural Model



Microservices Application Architecture: An Example



Comparison Monolithic vs Microservices Architecture

	Monolith	Microservices
Scalability	Entire app should be re-deployed to release new changes / updates	New services can be deployed without affecting the entire app
Database	An app uses a shared database	Every service uses a database
Development Team	Single team of SW engineers should develop, test, & maintain an app	Separate teams are allocated to develop, test, & maintain services
Technologies	Developers are required to use a defined tech stack	Every service can be developed by using different technologies

Comparison Monolithic vs Microservices Architecture ... cont'd

MONOLITH AT SCALE



Problem: Large single codebase

It is hard to maintain and update for one or several software engineers.



Problem: Outdated technologies

Developers have to stick to a defined tech stack when developing new features.



Problem: Slow development and testing

Time-consuming development of new functionality and comprehensive app testing. The entire app has to be re-deployed.

MICROSERVICES



Solution: Independent codebases

Dedicated teams of developers can easily maintain and update small codebases.



Solution: Individual tech stacks

Software engineers can incorporate new technologies when developing new microservices.



Solution: Autonomous team for each service

Each microservice is developed, tested, and deployed individually. Each team of developers can work independently.

Comparison Monolithic vs Microservices Architecture ... cont'd



Problem: More bugs

The entire application can hardly be tested by a QA team. More complex architecture leads to more bugs.



Solution: Efficient testing

Each service is tested individually, so there is no need to test the entire app. Small services are easier to test.



Problem: High failure chance

A database failure or a break in one line of code can crash the entire system.



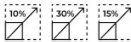
Solution: Fail-proof design

The entire system remains unaffected if one microservices fails.



Problem: Complete system scaling

The entire system should be scaled to achieve performance improvements.

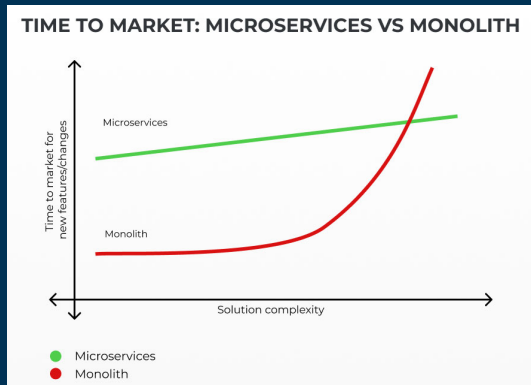


Solution: Independent scaling

Each service can be scaled individually, following their unique performance requirements.

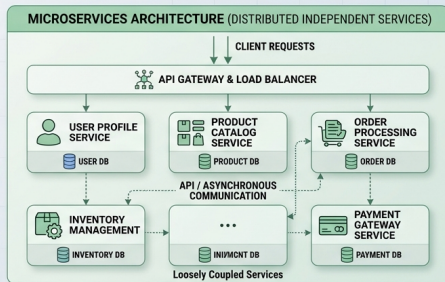
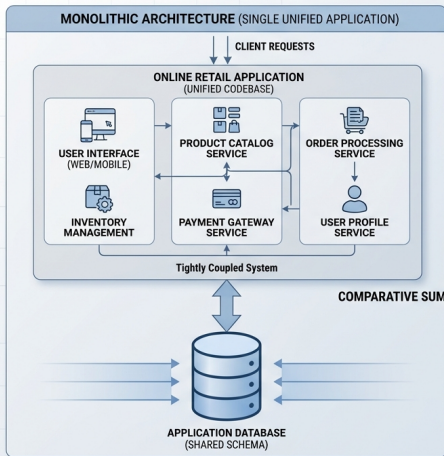
Mono vs Micro

- Many applications have the monolithic architecture because it foresees the opportunity to develop and launch an app with basic functionality quickly.
- Comparison of time required to launch an app with the microservice vs monolith architecture is shown in below graph.



Mono vs Micro ... cont'd

COMPARATIVE ANALYSIS: MONOLITHIC VS. MICROSERVICES ARCHITECTURE FOR SOFTWARE DESIGN AN EXAMPLE: ONLINE RETAIL PLATFORM



COMPARATIVE SUMMARY: KEY CHARACTERISTICS

CHARACTERISTIC	MONOLITHIC	MICROSERVICES
CODEBASE & DEVELOPMENT	Single, complex codebase	Small, focused services
DEPLOYMENT	All-or-nothing, entire application	Independent service deployment
SCALING	Scale entire application	Scale specific services
DATA MANAGEMENT	Single shared database	Database per service (Polyglot persistence)
FAULT TOLERANCE	Single failure affects all	Failure isolation, localized impact
TECHNOLOGY CHOICES	Unified technology stack	Freedom to use different stacks
TEAM STRUCTURE	Large team, coordinated releases	Small cross-functional teams, rapid delivery

Key Characteristics of Microservice Apps

Due to the distributed architecture, microservices applications have distinctive characteristics as follows:

- Independent components

- One of the primary benefits of the microservice architecture over the monolith is the use of independent components.
- They are responsible for completing certain tasks only. Distinctive microservices don't affect other components of an application.

- Decentralized approach

- Each microservice has its database and data processing unit. Therefore, there is no need to maintain an expensive centralized database and server to process and store all the information.
- Moreover, the decentralized product development approach offers the opportunity to apply various technologies to achieve certain business goals.

Key Characteristics of Microservice Apps ... cont'd

- Great evolutionary

- The benefits of the microservice architecture foresee the opportunity to upgrade applications by adding new features fast.
- Consequently, applications that use microservices can quickly grow and obtain new functionalities.

- Failures resistance

- Since each microservice is an independent part, it can be isolated or disabled.
- In such a case, the entire application may lack certain functionality. However, it will keep working without any issues.
- The updated microservice can be connected back after fixing an error.

Benefits of Microservices Architecture

The benefits of the microservice architecture help businesses develop reliable, scalable, and easy-to-maintain applications fast.

Advantages of the microservice architecture are listed below.

- Self-sufficiency
- Technical variety
- Scalability
- High sustainability
- Continuous delivery
- Easy changes implementation
- Simplified onboarding

Problems solved by Microservices

Microservice architecture helps solve a lot of problems for large businesses and enterprises that grow fast, giving tech companies reasons solving following problems:

- **Complex Codebase:** Monolithic applications that constantly evolve by adding new features have complicated codebases. The use of microservices helps software engineers to reduce the complexity of an application's codebase.
- **Poor Scalability:** Provide an opportunity to scale up an app fast and stress-free. Moreover, businesses can develop and connect new features to evolve their applications.
- **Unacceptable Downtime:** Outstanding failure resistance and the absence of the need to redeploy an application to release new apps are the key benefits of using the microservice architecture. They help significantly increase an app's uptime to cut business losses.
- **Deficient Technology Stack Capabilities:** Some technologies may not offer the required tools to solve business problems. The use of microservices helps overcome this challenge because software engineers can create microservices using various technologies and connect them.